ZAMZAM FASHION · AUDIT & DELIVERY PLAN

From prototype to a production store

A full review of the code we received, what is broken, what is worth keeping, and a step-by-step plan to launch a secure, bilingual fashion and tailoring store.

> **Verdict:** the current build is a good-looking **clickable demo**, not a store. Orders never reach the owner, the API does not compile, and the two halves do not match. We keep the brand, the content and the tailoring know-how, give it a new UI, and rebuild the engine properly.

Prepared by Youssef Gamal

Date 5 November 2026

Scope D:\New_Downloads\Zamzam-Fashion (41 files)

Version 1.3 · 47 APIs · Vercel + MonsterASP.NET + Supabase

Zamzam Fashion

01 · EXECUTIVE SUMMARY

Where the project stands today

The client asked for a bilingual (Arabic / English) online store that sells ready-made clothes and takes made-to-measure tailoring orders. The folder contains two separate pieces that were never joined together.

0%	1	32	5
FRONTEND ↔ API CONNECTED	COMPILE ERROR IN API	ISSUES FOUND	DAYS UNTIL .NET 8 END OF SUPPORT

What works

- A polished visual identity: beige, navy and gold, Arabic-first, RTL / LTR switch.
- Clear customer flows: browse, pick size and color, cart, cash-on-delivery checkout.
- A detailed tailoring studio: 6 garment families, 12 styles, about 45 measurement fields.
- An admin area for products, categories, orders and site content.
- Backend ideas worth reusing: strong password hashing, server-side prices, price snapshots, quote approval and a status workflow.

Why it cannot go live

- All data lives in each visitor's browser (`localStorage`). A customer's order stays on the customer's phone, so **the owner never receives it**.
- The admin "login" is a passcode checked in the browser and printed on the login page.
- The API fails to build (`CS8110`) and has a high-severity vulnerable package.
- The API's data model cannot store sizes, colors, descriptions, sub-categories or the tailoring fields the frontend collects.
- Product photos are AI-generated placeholders, and the same 9 images are reused across 15 products.

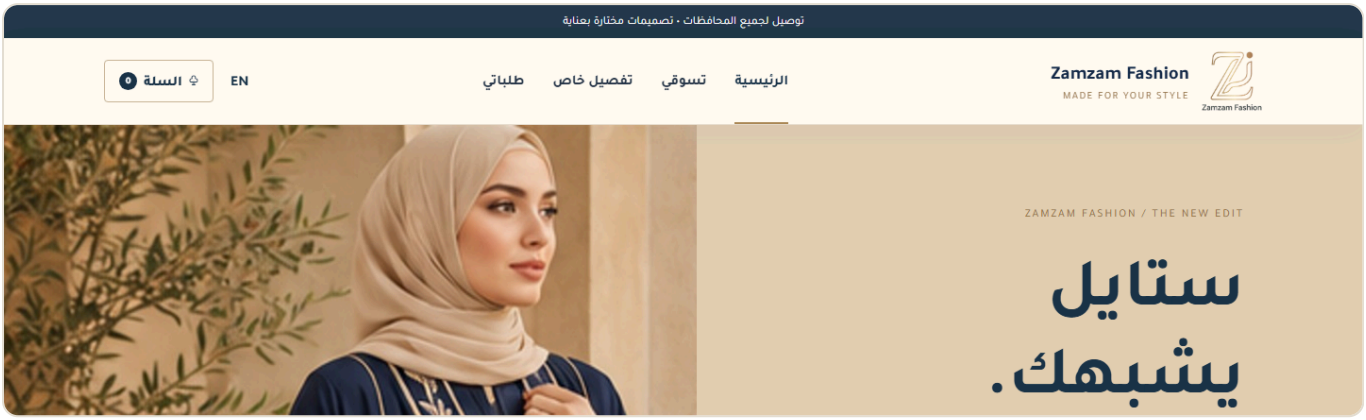
Recommendation. Treat the current build as a **requirements prototype**. Rebuild it on a real stack: an **ASP.NET Core 10 API** with PostgreSQL, a **newly designed Next.js storefront and admin**, and cloud image storage. Then launch a **cash-on-delivery MVP in 7 days** and add online payments and courier integration in phase 2. Patching the demo would cost more than rebuilding, because the browser-only design runs through every function.

How we reviewed it

- Read every file: 66 KB `app.js`, 40 KB CSS, the API (`Program.cs`, `Models.cs`), 15 release notes and the launch scripts.
- Built the API with the .NET SDK in an isolated copy, and ran `dotnet list package --vulnerable`.
- Rendered the storefront in a headless browser and traced each admin and checkout function by hand.
- Compared the frontend's data shapes with the API's models and contracts, field by field.

02 · INVENTORY

What we received



Current storefront home page (Arabic). The UI will get a new design; the logo and brand colors stay unless the client wants a change. The hero image is AI-generated, not a real Zamzam product.

Part	Tech	State
frontend-preview/ index.html, app.js, styles.css, 12 images	Vanilla JS, one 66 KB file, rendered with `innerHTML`	Runs offline with Python's static server. Saves everything to the browser's localStorage. It does not call the API at all.
backend/Zamzam.Api/ Program.cs (46 lines), Models.cs	ASP.NET Core 8 Minimal API, EF Core, SQLite / SQL Server, JWT	**Does not compile.** No migrations, tests or image uploads. The README says it "has not been compiled in this environment".
15 notes files CHANGELOG v0.3 to README v1.8	Markdown / txt	They contradict each other: ports 5500, 5513 and 5524, and a promised Angular app that does not exist. Each one repeats "demo only, not production".
Launch scripts	.bat	Need Python and the .NET SDK on the client's PC. These scripts are not a deployment.
Brand assets	PNG / JFIF	Logo files are 535 KB and 669 KB, about 20 times too heavy for the web. We still need the vector (SVG) source.

Feature map: what the demo shows vs. what the API supports

Feature	Demo UI	API	Production need
Catalog, categories, sub-categories	Yes (local)	Partial, no sub-categories	Yes + slugs, SEO
Sizes, colors, stock per variant	Size/color, stock per product	None	Stock per size × color
Product description, multiple photos	1 photo, base64	1 URL	Gallery on CDN
Guest checkout, cash on delivery	Yes (local)	Login required	Guest + phone
Delivery fee	One flat fee	None	Per governorate
Tailoring request + reference photo	Yes (local)	Different fields, no photo	Yes, unified
Quote → customer approval	No	Yes	Yes, via secure link
Order tracking for the customer	Same device only	Logged-in users	Tracking link / phone
Admin authentication	Fake passcode	JWT role	Identity + lockout
Site content and footer settings	Yes (local)	None	Yes
Notifications to owner and customer	No	No	Email + WhatsApp

03 · AUDIT FINDINGS

What is broken, and how badly

Each finding was verified in the code or by running it. Severity is based on launch impact: CRITICAL blocks launch, HIGH breaks a core flow or creates real risk, MEDIUM hurts quality, security depth or performance, and LOW is hygiene.

ID	Severity	Finding	Evidence
C1	CRITICAL	**The API does not compile.** A local function is used inside an EF Core query.	`Program.cs(31)` error CS8110 in `/api/orders/mine`
C2	CRITICAL	**Orders never reach the store.** Products, carts and orders are saved only in the visitor's own browser, and the frontend makes no API calls.	`app.js:12` `localStorage`, key `zamzam_demo_v2`; no `fetch` anywhere
C3	CRITICAL	**No real admin security.** The passcode is checked in client JavaScript and shown on the login screen.	`app.js:86-87` "Demo code: zamzam-demo"
C4	CRITICAL	**The runtime is about to lose support.** .NET 8 support ends on 10 Nov 2026, so there will be no more security patches.	`Zamzam.Api.csproj` → `net8.0`
C5	CRITICAL	**Placeholder product imagery.** The images are AI-generated, 9 photos are reused across 15 products, and the prices and stock are invented.	`IMAGE-GUIDE.md`, seed data `app.js:3-11`
H1	HIGH	**The API's data model cannot hold the catalog.** It has no sizes, colors, variants, description, gallery, parent category or settings.	`Models.cs` Product has 8 fields
H2	HIGH	**The tailoring contracts don't match.** The UI has 6 garment types with about 45 field keys. The API has 4 types with different keys (`shoulders` vs `shoulder`, `tops` vs `blouse`) and rejects any unknown key, so almost every real request would fail.	`app.js:46-56` vs `Program.cs:33`
H3	HIGH	**The two sides use different status lists.** The admin UI can jump to any status. The API does not restore stock when an order is cancelled.	`app.js:84,115` vs `Program.cs:43`
H4	HIGH	**Checkout rules conflict.** The UI is guest checkout, but the API requires an account. The API also does not store the customer's name, governorate, city or delivery fee.	`ReadyOrderRequest`
H5	HIGH	**Stock is counted per product, not per size and color.** The store can sell an "M / Beige" that doesn't exist.	`Product.Stock`, `addCart()`
H6	HIGH	**A dependency has a known vulnerability:** `SQLitePCLRaw.lib.e_sqlite3 2.1.6` (High).	GHSA-2m69-gcr7-jv3q
H7	HIGH	**There is no image storage.** Uploads are base64 strings in localStorage (about a 5 MB browser limit), and the API has no upload endpoint.	`createProduct()`, `submitCustom()`
H8	HIGH	**Editing a product always fails** unless the admin re-ticks every size and color. The form never pre-fills them, because `setAdminChecks()` is defined but never called.	`app.js:98,157,164`
H9	HIGH	**There are no database migrations.** The schema is built with `EnsureCreated()`, so any later change means losing data.	`Program.cs:10`
M1	MEDIUM	Authentication is thin: 8-hour JWTs with no refresh or revoke, no lockout or rate limit, and no password reset or verification.	`Token()`
M2	MEDIUM	Admin endpoints are missing: delete product or category, edit category, settings, uploads, customers.	API endpoint list
M3	MEDIUM	The production config is wrong: `AllowedHosts: localhost`, CORS hard-coded to localhost, no HTTPS/HSTS, no error handler, logs or health check.	`appsettings.json`, `Program.cs:8-9`

ID	Severity	Finding	Evidence
M4	MEDIUM	The frontend has no SEO and is fragile. It has no product pages or real URLs. The browser Back button is broken (`replaceState`). It uses 38 inline `onclick` handlers (which block a CSP) and re-renders the whole page on every click.	`app.js:20,167`
M5	MEDIUM	XSS risk. Escaped values are placed inside inline JS strings, and the browser decodes the entities back into quotes. The order image `href` is not escaped.	`app.js:81` removeCart, `:108`
M6	MEDIUM	The "Banner headline" setting is saved but never shown on the site.	`bannerAr` is only read in the settings form
M7	MEDIUM	The pages are heavy on mobile data: a 0.67 MB logo on every page and two font imports (3 families), with no WebP or responsive images.	`styles.css:1-2` , assets/
M8	MEDIUM	There is one flat delivery fee (65 EGP) for all of Egypt. Couriers charge by governorate.	`settings.deliveryFee`
M9	MEDIUM	There are no notifications. Neither the owner nor the customer gets an email or WhatsApp message when an order is placed.	—
M10	MEDIUM	The order flow uses no transaction isolation or concurrency token, so two buyers can oversell the last item.	`/api/orders/ready`
L1	LOW	There is dead code: `editProduct` , `changeProductPhoto` , `cancelProductEdit` and `setAdminChecks` are never used.	`app.js`
L2	LOW	There are 15 contradictory release notes. The ports change between versions, and the README promises Angular.	Root folder
L3	LOW	There is no git history, tests, CI, linting or `.env` documentation.	—
L4	LOW	There are no legal pages (privacy, returns and exchange, terms). Egyptian consumer and data-protection law expects them.	—
L5	LOW	Search, sorting, pagination and product detail pages are missing.	—
L6	LOW	Order IDs come from `Date.now()` , so they are guessable and can collide. Real stores need a readable number (ZF-10234) plus a private tracking token.	`app.js:20`
L7	LOW	Emoji fallbacks and a duplicate Google Fonts import look unprofessional when images fail.	`card()`
L8	LOW	Swagger is on by default in Development, and there are no API versioning or ProblemDetails responses.	`Program.cs:9`

04 · SALVAGE

What we keep, rework and drop

The client already paid for good design and domain thinking, so we keep that work and replace what cannot be fixed.

Keep as-is

- Brand logo and colors (the UI layout gets a new design)
- All Arabic and English copy and labels
- Tailoring catalog: 12 styles, 6 garment families, measurement groups
- Color list with hex swatches
- Customer flows: variant picker, cart and checkout steps
- PBKDF2 hashing with 210k iterations (via ASP.NET Identity)

Rework

- The data model: rebuild it around variants, a gallery and delivery zones
- The order status machine: one list shared by the UI and API, plus history
- Custom pricing: an instant calculator, then the owner confirms or adjusts
- Server-side pricing and price snapshots: keep, and add size and color
- The whole UI: a new design for the storefront and the admin

Drop

- localStorage as the database
- The browser passcode "login"
- The single-file string-template rendering and inline handlers
- `EnsureCreated` and the SQLite production path
- The .bat launchers and the 15 release notes (replace them with one README)
- AI placeholder photos once real ones arrive

05 · TARGET ARCHITECTURE

How the new system fits together

Customer
Phone browser · AR / EN

Owner (runs the shop and does the tailoring)
Admin dashboard · one account, ready for staff later

▼

Vercel (our company's paid plan)
Next.js site · CDN · free SSL · forwards `/api/*` to the API (first-party login cookie)

▼

Next.js on Vercel
Storefront with server-side rendering for SEO and fast first load · `/admin` dashboard · next-intl for RTL/LTR · new UI design

ASP.NET Core 10 API on MonsterASP.NET
Clean Architecture · Controllers · Identity httpOnly cookie · FluentValidation · rate limiting · Serilog · health checks

▼

Supabase PostgreSQL
Own schema `zamzam` · Data API off · EF Core migrations · nightly backup

Supabase Storage
Public bucket for product photos · private bucket for tailoring photos · WebP 400/800/1600

Notifications
Email (Resend / Brevo) · WhatsApp link → Business API later

Phase 2
Paymob (cards, wallets, Fawry) · Bosta courier API

Decision	Choice	Why
Backend	ASP.NET Core 10 · Controllers · Clean Architecture	LTS to Nov 2028; business rules stay testable.
Frontend	Next.js + TypeScript	Server rendering: product pages rank on Google and preview well on WhatsApp.
Database + photos	Supabase (PostgreSQL + Storage)	Managed PostgreSQL with `jsonb`; one dashboard for data and photos.
Hosting	Vercel (site) · MonsterASP.NET (API)	No servers to manage. All three services in one region keeps every request fast.
Payments (MVP)	Cash on delivery only	This matches the Egyptian fashion market and the client's current flow. Paymob comes in phase 2.

06 · SCOPE

What the MVP launch includes

Storefront

- Home page with hero, categories, new collection and featured picks (all editable)
- Shop with category and sub-category filters, sort, search and pagination
- **Product page** with gallery, size and color picker, live stock per variant, size guide and description
- Cart that is saved and checked against server stock and price
- Guest checkout with governorate-based delivery fee and cash on delivery
- Order tracking with order number + phone, or a private link
- Arabic / English with correct RTL, on mobile first

Made-to-measure

- Choose a style from the 12 models or "my own design"
- Garment, fabric, color and fit, with the measurement groups from the demo
- Body measurements (cm) or a ready size
- Upload a reference photo (checked, resized, private)
- **Instant price estimate**; the owner confirms or adjusts, and the customer re-approves only if the price changed
- Production stages: approved → in production → quality check → packing → shipped

Admin dashboard

- Secure owner login (one account, ready for staff later)
- Products with variants (size × color stock), multi-photo upload and show/hide
- Categories with sub-categories and sort order
- Orders: filter, details, price confirmation, valid status changes only
- Delivery fees, site content, contacts and tailoring price rules
- Dashboard numbers: new orders, pending quotes, low stock

Operations

- Email to the owner for every new order or tailoring request
- Email or WhatsApp click-to-chat confirmation to the customer
- Order status history: who changed what and when
- Automatic daily database and image backups
- Error tracking and uptime alerts
- Privacy, returns and exchange, and terms pages

Phase 2 (after launch): AI tools (design preview, size helper, style chat, owner assistant), Paymob payments, Bosta courier API, staff accounts and audit log, customer accounts, coupons, wishlist, reviews and WhatsApp Business API.

07 · DATA MODEL

Core entities

Entity	Key fields
Category	ParentId, NameAr/En, Slug, SortOrder, ImageUrl, IsVisible
Product	CategoryId, NameAr/En, Slug, DescriptionAr/En, BasePrice, IsVisible, CreatedAt
ProductImage	ProductId, Url, AltAr/En, SortOrder
ProductVariant	ProductId, Size, ColorId, Sku, Stock, PriceOverride?, *RowVersion* (prevents overselling)
Color	NameAr/En, Hex
Order	Number (ZF-10234), TrackingToken, Type (Ready / Custom), Status, CustomerName, Phone, Governorate, City, Address, Notes, Subtotal, DeliveryFee, Total, PaymentMethod, PaymentStatus
OrderLine	VariantId, snapshots: name, size, color, unit price · Qty
TailoringSpec	OrderId, StyleId, Garment, Fabric, Color, Fit, SizingMode, StandardSize, Measurements (jsonb), ReferenceImageKey, EstimatedPrice, QuotedPrice, QuoteApprovedAt
OrderStatusHistory	OrderId, From, To, ChangedBy, ChangedAt, Note
DeliveryZone / SiteSettings / PriceRules	Governorate → fee; editable content; base price per style + fabric, lining, size and extras

08 · ROADMAP

Delivery plan: 7 days, backend first

Days 1–4 build the full API (47 endpoints, listed in `Zamzam-API-Plan.docx`). Days 5–6 connect the new UI design to it. Day 7 is QA and launch. Each day ends with a working build on staging and a short demo.

BACKEND
Day 1

Foundations and owner login

Morning: a 45-minute kick-off call to confirm the open decisions and collect the catalog, photos, logo, delivery fees and price rules. **Rest of the day:** the .NET 10 Clean Architecture solution (Domain, Application, Infrastructure, Api, Tests), the Supabase database (own `zamzam` schema), all entities and the first migration, the Identity cookie login for the Owner, and CI on GitHub.

Done when the Owner can log in, the schema is migrated and CI is green. APIs #1–4.

BACKEND
Day 2

Catalog API

The public catalog (categories, products, product page, site settings). Admin categories, products, variants (stock per size × color), colors, and the image upload pipeline: type check, resize to WebP with SkiaSharp, store in Supabase Storage. Plus validation, ProblemDetails errors and Serilog. Seed the real catalog.

Done when Swagger shows the real catalog. APIs #5–8 and #17–34.

BACKEND
Day 3

Orders API

Delivery zones, then guest checkout with server-side re-pricing, stock reservation with a concurrency token and an idempotency key. Then the order number and tracking token, the tracking endpoint, the status machine with history and restock on cancel, the admin order board, and emails to the owner and customer.

Done when two buyers can't both get the last item, and the owner gets an email within a minute. APIs #9–11, #35–37, #41–42.

BACKEND
Day 4

Tailoring, pricing and settings API, then hardening

The tailoring catalog, the price rules and the instant estimate, tailoring requests with a private photo, price confirmation or adjustment, and customer approval or rejection. Site settings and the dashboard numbers. Then rate limits, security headers, health check, integration tests on real PostgreSQL, and deploy to staging.

Done when all 47 APIs are on staging and their tests pass. APIs #12–16, #38–40, #43–47.

FRONTEND
Days 5–6

New UI, connected to the API

A Next.js app with the **new design** (Arabic/English, RTL, mobile first). **Day 5:** the storefront: home, shop, product page, cart, checkout, tracking, and the tailoring form with the live price estimate. **Day 6:** the owner dashboard: products and stock, photos, orders, price confirmation, price rules, delivery fees and site content. SEO: metadata, sitemap and Open Graph.

Done when a customer can order from a phone and the owner can run a full day alone.

QA + LAUNCH
Day 7

Hardening, QA and launch

Morning: security pass, Playwright E2E tests in AR and EN, real Android and iPhone tests, a backup restore test, and the legal pages. **Afternoon:** production deploy, monitoring, soft launch, then public launch, and owner training.

Done when there are no open Critical or High issues, real orders are flowing and the handover is signed. Two weeks of support follow.

What makes 7 days possible. (1) The client delivers **all content on Day 1**. (2) The **UI design is ready by Day 5**. (3) Scope is **frozen** at 47 APIs, and new ideas go to phase 2. (4) Full-time, AI-assisted development. If content or the design is late, the launch moves by the same number of days.

09 · DEFINITION OF "PRODUCTION-READY"

The launch checklist

We go live only when every box is ticked. This list is also the client's acceptance criteria.

Security

- [ ] Server-side owner login, lockout and rate limits
- [ ] HTTPS everywhere with HSTS and a strict CSP
- [ ] Secrets in environment variables, never in git
- [ ] Validation on every input, server-side prices only
- [ ] Uploads: type sniffing, size cap, re-encoding, private bucket for tailoring photos
- [ ] No vulnerable packages (checked automatically in CI)

Data

- [ ] EF Core migrations, no `EnsureCreated`
- [ ] Automated daily backups kept 30 days, with one tested restore
- [ ] Concurrency tokens on stock
- [ ] Order status history (who, when, from, to)
- [ ] Customer data only as needed (Egyptian Law 151/2020)

Performance and SEO

- [ ] Mobile Lighthouse ≥ 90 for performance, SEO and accessibility
- [ ] WebP/AVIF images, responsive sizes, CDN caching
- [ ] Logo as SVG (smaller than 10 KB)
- [ ] Clean product URLs, sitemap, hreflang, Open Graph and schema.org
- [ ] Google Search Console and Analytics connected

Quality

- [ ] Unit tests: pricing, stock, status transitions, measurements
- [ ] Integration tests against real PostgreSQL (Testcontainers, in GitHub Actions)
- [ ] E2E tests in AR and EN for checkout, tailoring and admin
- [ ] Manual RTL pass on Android Chrome and iPhone Safari
- [ ] Accessibility: keyboard use, contrast, labels

Operations

- [ ] CI/CD: tests, then staging, then production with one click
- [ ] Error tracking (Sentry) and uptime alerts
- [ ] Structured logs kept 14 days
- [ ] One README with runbook and env variables
- [ ] Rollback procedure written and tested

Business and legal

- [ ] Real product photos, prices and stock
- [ ] Delivery fees per governorate confirmed
- [ ] Returns and exchange policy (Consumer Protection Law 181/2018), privacy and terms
- [ ] Business email on the domain, WhatsApp number
- [ ] Owner trained, and the handover signed

The legal references are for guidance only. The client should confirm the final policy wording with their accountant or lawyer.

10 · NEXT STEPS

What we need from the client

Content and assets (needed on Day 1)

- Logo source file (SVG / AI / PDF) and brand fonts, if any
- Real product photos: at least 3 per product on a plain background
- Product sheet: name AR/EN, category, price, sizes, colors, stock per size × color, fabric and care
- Delivery fee per governorate, plus delivery times
- Tailoring: confirmed styles, fabrics, the required measurements per garment, and the price rules (base price per style + extras)
- Contacts: phone, WhatsApp, email, address, social links
- Policy text: returns, exchange and privacy (we can draft templates)

Accounts (owned by the client)

- Domain name (for example zamzamfashion.com)
- Production Supabase project and MonsterASP.NET account (we are invited as admins); the domain pointed to Vercel
- Google Search Console and Analytics
- Phase 2: Paymob merchant account and Bosta business account (these need a commercial register / tax card)

Decisions

Question	Decision	Status
Who uses the dashboard?	One Owner account (the owner works alone); roles stay ready for staff later	**Decided**
Customer accounts?	Guest checkout + private tracking link	**Decided**
Ready-made vs custom?	Both equally important	**Decided**
Custom-order pricing	Instant calculator from price rules; the owner confirms or adjusts	**Decided**
Payments · AI tools	Cash on delivery only · all AI ideas in phase 2	**Decided**
Deposit for tailoring?	Recommended: manual InstaPay deposit for high-value orders	Open
Delivery · language	Recommended: manual courier at launch · Arabic by default	Confirm

Risks and how we handle them

Risk	Mitigation
Real photos and catalog arrive late	This is the main launch blocker. Development continues with placeholders, but the launch date moves with the content. Content is due on Day 1, and each day it is late moves the launch by one day.
Scope creep (new features mid-build)	Scope is fixed per phase. New ideas go to the phase 2 backlog, which is reviewed at each evening demo.
Tailoring rules are unclear	A 1-hour session with the owner (who is also the tailor) on Day 1. The measurement catalog is data in the database, so it can change without new code.
COD refusals (fake orders)	Phone confirmation status before shipping, a block list by phone, and an optional deposit for high-value orders.
A hosting service goes down	Managed services with uptime alerts, a nightly database backup kept outside Supabase, and API redeploys from GitHub in minutes.

Immediate next step: a 45-minute call with the client to walk through this plan, confirm the two open decisions above, and set the content deadline. Day 1 of the build starts right after it.